AN2DL - First Challenge Report
Gr4dient G4ng

Paolo Ginefra, Martina Missana, Lorenzo Pettenuzzo, Matteo Romilio Pasqual
paologinefra, martinamissana1, pettenuzzolorenzo, matteopasqual02
278153, 274235, 273597, 274006
November 17, 2025

1 Introduction

This project addresses the first challenge of the *Artificial Neural Networks and Deep Learning* course (A.Y. 2025–2026), which focuses on *multivariate time series* classification. The task consists of predicting pain intensity levels from a collection of temporal signals.

The dataset is composed of n time series of fixed temporal length $T = 160$. For each sample $i \in \{1, \dots, n\}$, the raw input is a tensor $X_i \in \mathbf{R}^{F \times T}$, where $F = 38$ denotes the total number of feature channels. These channels include: 4 survey-derived channels, 31 biomechanical channels, and 3 global features. Each multivariate time series has a categorical label $y_i \in \{0, 1, 2\}$, corresponding to *No Pain*, *Low Pain*, and *High Pain* and the goal is to predict the labels of unseen data.

2 Problem Analysis

The problem presents several key challenges.

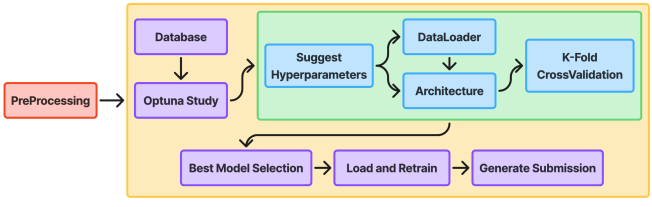
The limited dataset size and class imbalance increases the risk of overfitting and biases predictions toward frequent labels, requiring regularization, careful hyperparameter tuning, and robust cross-validation.

Our approach is based on several assumptions. Pain levels are primarily reflected in temporal dynamics. Global static features, may provide priors that influence pain prediction and should be integrated with dynamic signals and dimensionality reduction techniques may capture dominant modes, reduce redundancy, and improve generalization.

Doing some initial analysis on the dataset, we realized that the signal 'joint_30' and physical information features were constant over time and the latters can be represented by a single variable. In addition some signal are very correlated reaching up to 99% of correlation for 'joint_10' and 'joint_11', had (Appendix A).

3 Methods

We aimed to build a software package, as generalized as possible, in line with the SOTA MLOPS frameworks. We built a fully automated pipeline that could be used to solve a general time-series classification problem. In our pipeline, Optuna manages the architecture and hyperparameter optimization, relying on a persistent database and allowing to merge heterogeneous computational resources in the same search experiment.



3.1 Pre-Processing

Let the raw dataset consist of $X \in \mathbb{R}^{N \times F \times T}$.

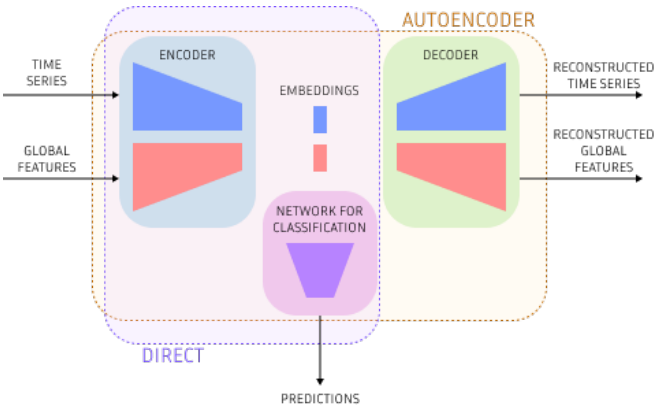
During the loading, each sample is $X_i = \{x_i(t)\}_{t=1}^{T} \in \mathbb{R}^{F \times T}$. Firstly we map physical informations to a binary pirate–indicator isPirate $\in \{0, 1\}$ and we drop the features we mentioned in section 2. After that, all continuous features are standardized using Z-score normalization and finally, global feature extraction computes a fixed-length

feature vector $g_i \in \mathbb{R}^D$, including statistical features (mean, variance, minimum, maximum, range, trend), domain-specific features (average, variance and divergence of the pain–survey) and PCA (applied independently to each trajectory $x_{i,f}$ keeping the 95% of the explained variance Appendix B).

Global features are refined through a multi-stage selection procedure, including Random Forest Importance, Mutual Information scoring and Lasso regression. The final subset of features is defined as the intersection of the top-ranked features across methods. Additionally, since in the train dataset the 'no_pain' label is dominant, to address class imbalance the class weights are computed using the inverse-frequency scheme: $w_c = \frac{N}{C\,N_c}$.

After we saw that the first experiments' performance translated poorly to the test evaluation, we decided to add a data augmentation module. We implemented time warping, jittering, scaling, offset and windowing. Windowing was not just used as data augmentation, but also to create an ensemble model over the windows for each sample.

3.2 Architectures



In our package the supported macro-architectures are Direct (Encoder + FFn) and Autoencoder (AE + FFn). The encoder may be RNN/LSTM/GRU, Conv1D (temporal convolutions with pooling), or MultiScaleCNN (parallel convolutional branches at multiple receptive fields). If enabled, the decoder mirrors the encoder. Finally, a feed-forward head performs classification.

Starting from the direct architecture (presented in the course) we moved to AEs to further utilize the limited provided data. In addition, the AE allows us to include the unlabeled test data (almost double the train ones) to help learn meaningful representations while keeping untouched the prediction loss.

3.3 Pipeline

Once the dataset is preprocessed, we setup the Optuna study, connecting all the available computational resources (e.g. local, Kaggle and Colab GPUs) to the same experiment in the Database. The optimization routine executes n_{trials} and for each, Optuna samples an architecture/hyperparameter configuration among the implemented ones, using a TPE sampler. A Median Pruner is also used to prune unpromising trials early, saving computational resources. This allows to reach faster a good configuration compared to the standard grid or Random search. The sampler also chooses whether to include the global features, to weight the loss with the class weights and to perform windowing.

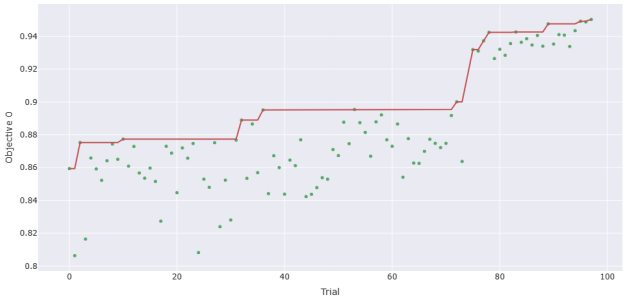
For each sampled configuration, Stratified K-fold cross-validation is applied with $K = 5$. For each fold, the pipeline (using the dataloader class) splits train and validation data, including the test data if the sampled macro-architecture is an AE and optionally loads the global features. The architecture is built using the sampled configuration and trained with AdamW to minimize a convex combination of the Reconstruction and the Prediction loss (MSE, cross-entropy). Stochastic Weighted Averaging (SWA) and Gradient Clipping (GC) are added to enache the training stability. We used Pytorch Lightning as a way to simplify model training, checkpointing and logging (using Tensorboard). After each trial the F1Score is computed averaging the best checkpoints for every fold.

When we decide to stop the study, the best model is chosen as the one that maximizes that metric, and the selected configuration (model + weights) is used to compute the predictions on the test data to generate the submission CSV. Optionally, we can retrain the best model and create an ensemble model where each sub-model is the best checkpoint for every fold.

4 Experiments

Experiments were conducted by defining a hyperparameter search space and letting Optuna select the configurations that maximized the F1-score across folds. Since the TPE sampler does not guarantee the finding of the global optimum, we repeated the optimization multiple times to increase the chances

of identifying the best configuration (Additional plots in appendix C).



Best Trial (number=97)

0.950271594524384

Params = [use_global_features: False, window_size: 32, jitter_strength: 0.0109404136591925, scaling_range: 0.140547790998147, time_warp_strength: 0.0869118049274283, architectureType: Recurrent, rnnType: GRU, hiddenDim: 158, RecurrentNumLayers: 3, recurrentDropout: 0.108890536590287, numFFLayers: 3, ffSizeCoefficient: 0.390358141099275, ffDropout: 0.0230767500272827, ReconstructionLossWeight: 0.0263093040933471, RegularizationWeight: 0.0025077476700133, useClassWeights: False]

5 Results and Discussions

The model that achieved the best performance on the leaderboard was an Autoencoder with Multiscale CNN (inspired by Inception [1]) achieving $F1Score = 0.9480$.

More in general, the experimental phase firstly shows that the Autoencoder consistently outperforms the Direct macro-architecture, benefiting from the added stability provided by the reconstruction loss and the bigger quantity of data. At the micro-architecture level, both MultiScaleCNNs and recurrent layers perform well, outperforming the Conv1D in the most of the cases. We suspect that it can be caused because standard Conv1D modules rely on fixed-size kernel and struggle to represent relationships at different frequencies.

The data augmentation was crucial to improve the performances of our models on the leaderboard. Regarding data augmentation, experiments showed better performance with smaller windows, suggesting minimal temporal patterns. In fact, flattening the data and performing classification using a linear feed-forward encoder achieved comparable results to more complex architectures.

Adding global features (such as mean, variance, ...) to the dataset did not substantially improve the prediction performance, the minority classes (especially High Pain) remained difficult to classify despite the usage of class-weighting strategies. The use of learning-rate schedulers reduced the sensitivity to the initial learning rate. Among the tested schedulers, Reduce-on-Plateau achieved the best performance, because it reacts to actual model performance instead of following a fixed schedule.

Overall, the system demonstrates solid generalization within the competition constraints, yet performance bottlenecks appear to be primarily linked to the limited expressiveness of the data rather than architectural insufficiency.

6 Conclusions

We presented our solution for this classification task. Although the software package is designed to be general, this generality results in a large hyperparameter search space. Given the time constraints and the associated curse of dimensionality, exhaustive exploration was not feasible.

The majority of our efforts have been spent in the development of a generalized infrastructure to perform efficient and large scale hyperparameter optimization. The main finding of this project is that architectures hyperparameters are essential but soon show diminishing returns. Our biggest gains have been achieved by implementing new forms of augmentations and their hyperparameters are consistently show the highest importance.

Despite this, the tool we have developed can be a formidable ally, given appropriate time constraint, to perform structured ablation study on proposed solutions.

To further improve our final performance much can be systematically tried, ranging from broader and better calibrated augmentation, to new architecture (attention based models being next in line) to learning optimizations. Although we achieved a high test F1-score (0.9480), we expect that with more time for hyperparameter exploration the performance could improve further.

Future work may include integrating attention-based encoders, enlarging the dataset, or augmenting it with synthetic expert-generated trajectories to better handle underrepresented classes.
[CODEBASE here]

References

[1] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. E. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. *CoRR*, abs/1409.4842, 2014.

Appendix

A Signal Correlation

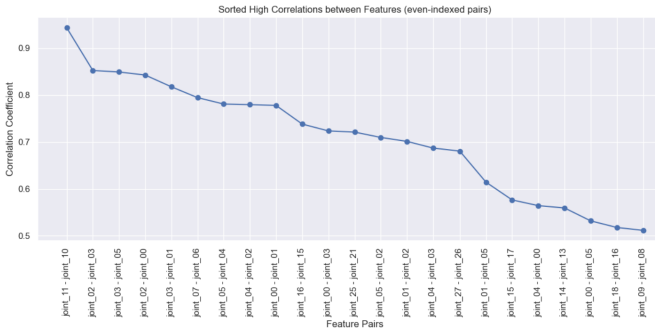


Figure 1: High correlated features plot

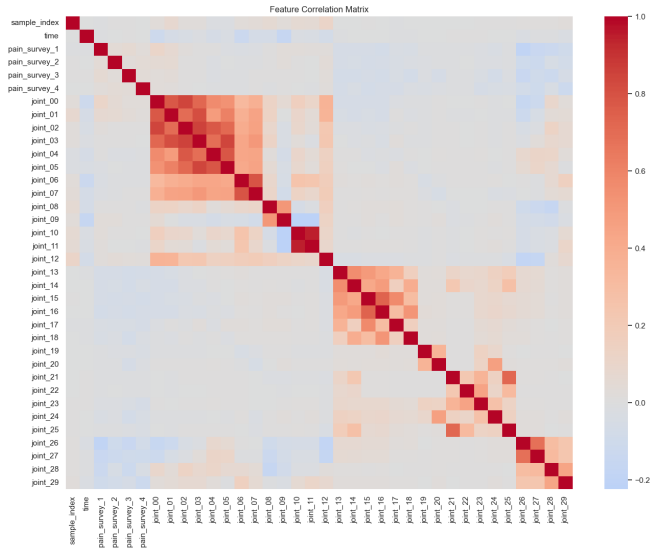


Figure 2: High correlated features matrix

B PCA

PCA is applied independently to each trajectory $x_{i,f} = (x_{i,f}(1), \ldots, x_{i,f}(T))^\top \in \mathbb{R}^T$. PCA yields $x_{i,f} \approx U_f z_{i,f}$, where U_f contains the leading eigenvectors and $z_{i,f}$ the corresponding POD coefficients. The number of retained components is the minimum k such that

$$\sum_{j=1}^{k} \lambda_{f,j} \geq 0.95 \sum_{j=1}^{T} \lambda_{f,j},$$

and the resulting POD coefficients are appended to the global feature vector.

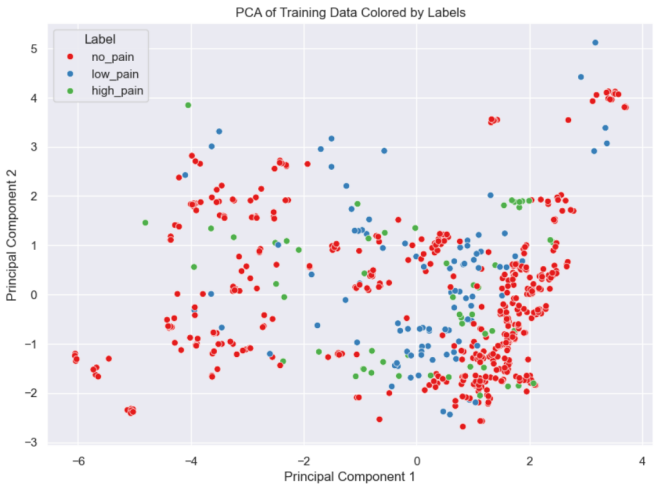


Figure 3: PCA on training data

C Experimental plots

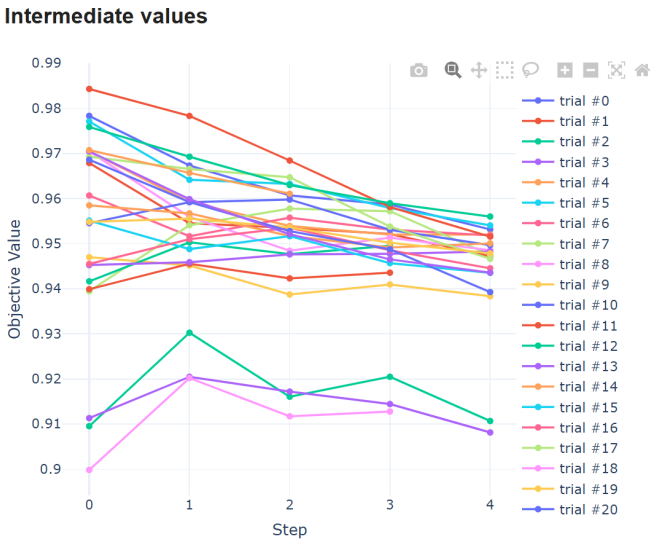


Figure 4: Fold F1Score

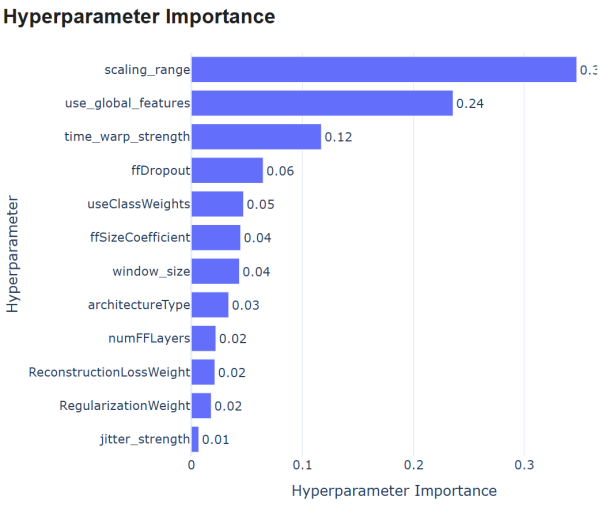


Figure 5: Hyperparameter importance

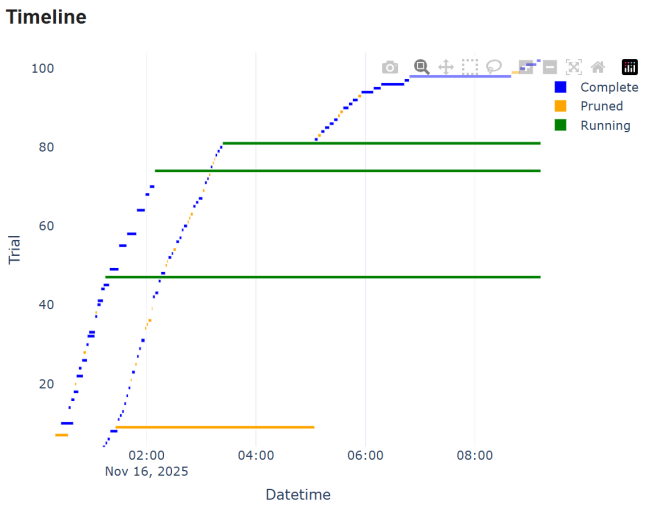


Figure 6: Timeline

D Results

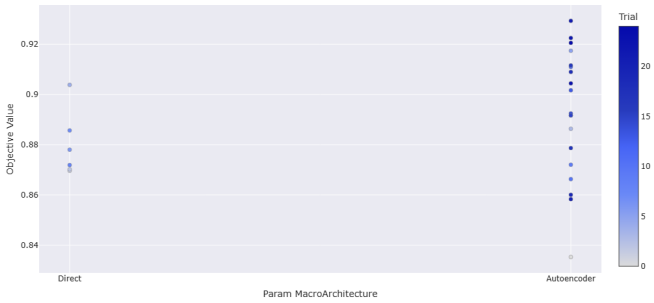


Figure 7: Autoencoder vs Direct performances